

LLMsVerifier

LLMsVerifier

验证。监控。优化。

概述

LLMsVerifier 是一款企业级平台，专用于验证、监控和优化跨多家供应商的大型语言模型（LLM），其核心基于强制性的“看见我的代码了吗？”验证测试，确保仅有经过实际验证的模型才会被标记为可用或导出。

简介

Go 平台可验证、基准测试、监控并优化多供应商的 LLM。每个模型在使用前必须通过强制性的代码可视性测试，随后进行延迟、流式传输、函数调用、视觉及嵌入检查，最终仅导出已验证的配置至 AI 与 CLI 工具。

详细说明

LLMsVerifier 是一款全面的平台，用于验证、监控和优化跨多家供应商的 LLM 性能。其核心原则是强制验证，且毫不妥协：任何模型在被标记为可用或允许导出配置之前，必须明确通过“看见我的代码了吗？”测试。该测试会向供应商发起真实 HTTP 请求，并分析响应内容，以确认模型是否真正理解输入，而非仅仅生成看似合理的回声。无法证明能“看见”并理解输入的模型，永远不会获得“可用”标记。通过这一关卡后，验证引擎将运行一系列全面的能力测试——包括存在性、响应性、延迟、流式传输、函数调用、视觉及 embeddings 检查，而报告引擎则将结果转化为可操作的 Markdown 和 JSON 报告。

该系统采用模块化、事件驱动架构，通过核心的验证引擎、报告引擎和配置管理器，提供 CLI、TUI、Web 及 REST API 接口。验证仅仅是起点，平台还集成了高级功能层，包括基于 LLM 的任务分解监管/工作模式、滑动窗口加 LLM 摘要的上下文管理（确保超长会话不中断）、云端检查点、配备断路器和基于延迟的路由的故障转移系统。周边基础设施则完全为生产环境设计：发布/订阅事件总线、定时调度、定价/限额检测、用于 RAG 的 vector 数据库及导出系统。平台采用独特的品牌标识约定，在每个生成的供

应商/模型名称后添加 (llmsvd) 后缀，确保已验证的输出一目了然，且绝不会与未经审核的模型混淆。同时，仅有通过验证的模型才会被写入导出配置，供 AI CLI 工具（如 OpenCode、Crush 和 Claude Code）使用。平台还提供团队在生产环境中实际需要的运维工具：Docker/Kubernetes/Helm 部署、Prometheus/Grafana 监控、LDAP/SSO 认证及 SQLCipher 加密存储。

为什么开发它

因为仅依赖配置检查并不可靠——API 密钥可能过期，模型可能被弃用，而配置文件无法反映真实延迟、真实错误，或模型是否真正“看见”并理解输入。LLMsVerifier 用实际证明取代“配置中有它，所以它一定能用”的假设：只有能够明确正确响应的模型，才会被标记为可用并导出。

内容

为何这是颠覆性的变革

它让 LLM 集群变得可信赖——这个词在充斥着隐瞒真相的配置的太空中极为罕见。团队不再寄希望于配置好的模型能正常运行，而是获得了一种强制执行、可测试的保证：每个投入使用的模型都经过真实验证，并通过监控、故障转移和仅验证模型导出机制，形成从验证到生产的闭环。在 Helix 生态系统中，它成为 LLM 模型、供应商及验证元数据的唯一真相来源：其他服务（包括 HelixTranslate）均以此为依据进行路由，整个平台因此拥有统一的答案——“当前哪些模型真正可用？”，而非各团队各自的乐观猜测。

创新之处

- 强制“看得见我的代码”验证——模型在可用之前必须通过一个真实的、基于 HTTP 的理解门槛；这是产品的核心差异化特征，也是确保未经验证的模型无法混入的关键。
- 仅验证模型配置导出——为 AI CLI 工具生成的配置文件仅包含已通过验证的模型，确保部署的配置不会悄然引入失效模型。
- (llmsvd) 品牌后缀系统——每个生成的供应商/模型均附带可追溯后缀，使验证来源在输出流转的每个环节清晰可见。
- 跨多种 CLI 代理和供应商的能力检测——它能识别流式传输类型（SSE、WebSocket、JSONL、EventStream）、压缩及缓存行为，而非一概而论。
- 弹性故障转移——断路器、基于延迟的路由（当首字节响应时间超过阈值时重新路由）、健康探测及加权流量分配机制，确保集群在单个供应商波动时仍保持响应。
- 长时自主运行——通过监督者/工作者分解模式、定期检查点存储及记忆集成，维持原本会耗尽上下文的长时间会话。
- RAG / vector-DB 集成，用于增强上下文基础。

最大的技术挑战及解决方案

- 证明模型真正可用，而非仅仅配置完成。这是核心目标，也是最棘手的难题。通过强制性的代码可见性测试解决——该测试会发起真实的 API 调用并分析响应以确认理解能力，辅以全面的能力测试套件，并拒绝导出任何未通过验证的模型，从而确保生产环节由验证而非配置决定。
- 应对众多不稳定的第三方供应商。通过故障转移协调器解决，将供应商不稳定视为常态：断路器在 N 次失败/M 秒内将供应商标记为退化，基于延迟的路由避开慢速端点，定期健康检查探测恢复情况，加权路由则在经济型与高端模型间取得平衡。
- 维持超长自主会话。通过监督者/工作者分解模式解决，将大型任务拆分为可管理的小块，定期检查点存储确保进度不因中断丢失，并采用分层上下文管理（滑动窗口 + LLM 摘要 + RAG），使模型在不被令牌淹没的情况下保持线程连贯。
- 供应商泛滥。通过将众多供应商特定的 Go 适配器隐藏在统一接口背后解决，实际端点由中心化枚举管理——添加供应商仅需局部修改，无需牵一发而动全身。

内容

技术栈

- **Go** —— 选为核心平台语言，因其具备高并发能力；驱动多线程验证引擎，可并行探测多个模型，并支持周边服务。
- **Gin** —— 选为 REST API 服务器，承载 JWT 认证、限流及 WebSocket/SSE 接口。
- **SQLite + SQLCipher** —— 选为嵌入式存储方案，支持数据库级加密，因验证数据（密钥、结果）敏感，需默认静态加密。
- **Redis** —— 选为缓存层，确保高频验证及元数据查询快速响应。
- **RabbitMQ + Kafka** —— 选为事件驱动架构的基础，通过消息与流处理实现平台内生产者与消费者解耦。
- **gRPC + Protocol Buffers** —— 选为强类型服务间通信及组件间事件传输方案。
- **QUIC / HTTP-3 (quic-go)** —— 选为现代传输协议支持（代码库文档提示 HTTP/3 提供商支持有限——仅为能力声明，而非普遍适用）。
- **JWT + LDAP/NTLM** —— 选为企业级认证方案，便于平台接入现有企业身份体系（文档宣称支持 SSO/SAML/OIDC）。
- **Viper**（配置）、**Logrus**（日志）、**Brotli/compress**（压缩） —— 运维基础设施：灵活配置、结构化日志及负载压缩。
- **Angular** —— 选为 Web 单页应用框架，作为验证与监控的可视化门户。
- **Python + JavaScript SDK** —— 选为客户端团队提供一流访问支持，并通过 OpenAPI/Swagger 文档化。
- **Docker、Kubernetes、Helm** —— 选为生产部署方案，支持健康监控与自动扩缩容，确保验证集群如现代服务般灵活伸缩。
- **Prometheus + Grafana** —— 选为指标与仪表盘方案，使平台自身健康状况与其监控的模型一样可观测。

- **Testify (Go) + node -test/jsdom (Web)** —— 选为分层测试框架，覆盖 Go 核心及 Web 前端。

状态与诚信说明

- 状态：测试版。Go 源码已实现真实 HTTP 验证（部分旧文档将验证描述为纯配置模式，属于愿景性描述，已过时——代码为准）。
- 许可证：待定。README 标注为 MIT，而 Dockerfile 标签为 Apache-2.0——发布前需明确。
- 适配器数量：README 声称“12 个适配器”，但 providers 目录实际列出约 26 个——应视为“12+ / 持续增加中”。多个标注为“FINAL/COMPLETE”的状态文件存在；以代码、文档及 go.mod 为准。
- 代码库位于 vasic-digital 组织下，但功能上属于 Helix LLM 基础设施集群的信任层。

优先级：Helix 主要（LLM 基础设施集群；LLM/提供商/验证元数据的唯一真实来源）。优先级次于 HelixTrack。